

P1837R0

Remove NTTPs of class type from C++20

Published Proposal, 2019-07-28

Author:

[Arthur O'Dwyer](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1837-remove-class-type-nttps.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1837-remove-class-type-nttps.html

Abstract

[P0732] "Class types in non-type template parameters" was accepted into C++20 in June 2018. Newly discovered issues with its premise, and new exploration in the area, suggest that we should postpone this feature until C++2b.

Table of Contents

- 1 A summary of P0732
- 2 P0732's killer app: template parameters of "string" type
 - 2.1 Slight variation remains ill-formed
 - 2.2 Subtle wording
- 3 Identity and equality are not the same thing in C++17

4 There is ongoing exploration of the NTTP space

4.1 `float` as NTTP

4.2 A mangling or serialization operator

5 Conclusion

Appendix A: Proposed straw polls

Appendix B: Proposed wording

References

Normative References

Informative References

This paper repeats some material from [\[Enums\]](#), and adds new material as well.

§ 1. A summary of P0732

C++ does not support non-type template parameters (NTTPs) of arbitrary types. For example, `template<int I>` is valid, but `template<std::string S>` is not valid.

Jeff Snyder's [\[P0732\]](#) "Class types in non-type template parameters" (first revision, 2018-02-11) observed that the essential problem preventing C++ from supporting arbitrary-typed NTTPs is that the compiler can't tell when two arbitrary NTTPs are "identical." "Identity" is important because it is how the compiler determines ([\[temp.type\]/1](#)) whether two templates are the same entity. For example:

```
using SomeType = [...];
template<SomeType X> void foo();
template<> void foo<SomeType(100)>() { return; }
template<> void foo<SomeType(356)>() { return; }
```

Here, if `SomeType` is `int`, then we have a valid program with two explicit specializations of `foo`. But if `SomeType` is `unsigned char`, then there is only one explicit specialization of `foo`, multiply defined; and so the program is ill-formed.

P0732 proposed that in present-day C++, determining the "identity" of two NTTP values is easy because the only supported NTTP types are simple scalar types where identity means equality, and where it is "obvious" to the compiler when two such values are equal. In C++2a, `operator==` can be defaulted; this increases the number of types for which it is "obvious" what identity means.

The rest of this paper will argue that "obviousness" is more subtle than we thought.

§ 2. P0732's killer app: template parameters of "string" type

Thanks to Louis Dionne for this example using P0732 NTTPs.

We create a function template `foo` that takes a `FixedLengthString<???` of deduced class type as its NTTP. (The CTAD syntax we're using here is homonymous with the syntaxes for plain old NTTPs and for concept-constrained type parameters, but it is not either of those.) When the programmer of `main` writes `foo<"hello">`, the compiler will perform overload resolution and class template argument deduction to determine that the right candidate is `foo<FixedLengthString<6>("hello")>`.

```
template<int N>
struct FixedLengthString {
    char data_[N] {};
    constexpr FixedLengthString(const char *p) {
        for (int i=0; i < N; ++i) {
            data_[i] = p[i];
        }
    }
    auto operator<=>(const FixedLengthString&) const = default;
};
template<int N> FixedLengthString(const char (&)[N]) -> FixedLengthString<N>;

template<FixedLengthString S>
int foo() {
    static int i = 0;
    return ++i;
}

int main() {
    int x = foo<"hello">();
    int y = foo<"hello">();
    return y;
}
```

`FixedLengthString<6>` has a structural comparison operator. Comparison on `FixedLengthString<6>("hello")` compares exactly the six bytes `"hello"` which are stored in `this->data_`. This program has well-defined behavior and returns 2.

My understanding is that this is the "killer app" for P0732 NTTPs: we can use literal class types to "smuggle" contraband such as string literals and floating-point values which would otherwise not be allowed in template arguments.

§ 2.1. Slight variation remains ill-formed

On the other hand, in this next example we create a function template `bar` that takes a concrete `VariableLengthString` as its NTTP. When the programmer of `main` writes `bar<"hello">`, the compiler will perform overload resolution to determine that the right candidate is `bar<VariableLengthString("hello")>`.

```
struct VariableLengthString {
    const char *data_ = nullptr;
    constexpr VariableLengthString(const char *p) : data_(p) {}
    auto operator<=>(const VariableLengthString&) const = default;
};

template<VariableLengthString S>
int bar() {
    static int i = 0;
    return ++i;
}

int main() {
    int x = bar<"hello">(); // ERROR

    auto& hello = "hello";
    int y = bar<hello>(); // ERROR
    return y;
}
```

`VariableLengthString` has a structural comparison operator, but when the compiler goes to mangle the name of `bar<VariableLengthString("hello")>`, it finds that it cannot produce a mangling of the `data_` member's value because `"hello"` is not a named variable. Therefore the program above is ill-formed. (The relevant wording is [\[temp.arg.nontype\]/2](#)).

The following `main` function would be well-formed:

```
int main() {
    static const char hello_array[] = "hello";
    auto& hello = hello_array;
    int y = bar<hello>();
    return y;
}
```

because `hello` refers to `hello_array`, and `hello_array` is a named variable.

§ 2.2. Subtle wording

The above-described behavior for template arguments involving pointers has been present since C++11 or earlier. It is quite subtle. It was implied by [\[N4700\]](#)'s old wording ([\[temp.type\]/1](#)):

Two *template-ids* refer to the same class, function, or variable if [...] their corresponding non-type *template-arguments* of pointer type refer to the same object or function or are both the null pointer value [...]

[\[P0732\]](#)'s changed wording, IMHO, obscures and possibly breaks the intent for template arguments involving pointers. [\[N4810\]](#)'s new wording:

Two *template-ids* refer to the same class, function, or variable if [...] corresponding non-type *template-arguments* have the same type and value after conversion to the type of the *template-parameter*, where they are considered to have the same value if they compare equal with the `==` operator [...]

The new wording does not clearly say what happens if the comparison with the `==` operator is not a constant expression. This can happen if it attempts to compare beyond-the-end pointers. [\[N4700\]](#) handled this because a beyond-the-end pointer does not "refer to an object." There is also relevant wording in [\[temp.arg.nontype/2\]](#), introduced in [\[N4268\]](#); but it does not mention beyond-the-end pointers.

Nor does the new wording say how the lookup for the `==` operator should be done. That's the next thing we'll look at.

§ 3. Identity and equality are not the same thing in C++17

Consider the following valid C++17 code:

```

enum E {
    ONE, TWO
};
namespace N {
    template<E> int foo() {
        static int i = 0;
        return ++i;
    }
    void test() {
        foo<ONE>();
        foo<TWO>();
    }
}

```

This code is valid and well-defined in C++17 today. It causes two distinct specializations of `foo` to be instantiated.

We can add the following overload of `operator==` anywhere in this code — before or after the definition of `foo`, inside namespace `N` or the global namespace — and the compiler won't care.

```

constexpr bool operator==(E, E) { return true; }

```

Thus, in C++17, it is perfectly possible that `static_assert(ONE == TWO)` and yet `static_assert(&foo<ONE> != &foo<TWO>)`. (Godbolt.)

With P0732 in the Committee Draft, two bad things happen. First, a wording issue: it's unclear whether, when [\[temp.type\]/1.5](#) asks if the arguments "compare equal with the `==` operator," the compiler will use our overloaded `operator==(E, E)`. (In practice, it will not.)

Second, because `E` is an enum type, it has strong structural equality ([\[class.compare.default\]/3](#)). So under P0732's rules we can create a literal class type `A` that can be used as an NTTP.

```

enum E {
    ONE, TWO
};
namespace N {
    struct A {
        E e_;
        bool operator==(const A&) const = default;
    };
    template<A> int foo() {
        static int i = 0;
        return ++i;
    }
    void test() {
        foo<A{ONE}>();
        foo<A{TWO}>();
    }
    constexpr bool operator==(E, E) { return true; }
}

```

Again, we can insert the overload of `operator==(E, E)` anywhere in this code — before or after the definition of `foo`, before or after the definition of `A`, inside namespace `N` or the global namespace.

It is not clear how `A::operator==` should call `operator==(E, E)`, if `operator==(E, E)` was not yet declared when `A::operator==` was defined.

No vendor has implemented P0732 NTTPs. But we can get some hint of the subtleties involved by looking at MSVC's in-progress implementation of `operator<=>`. ([Godbolt.](#))

§ 4. There is ongoing exploration of the NTTP space

[\[P0732\]](#) has done a great service by stimulating new exploration of the NTTP space. After P0732 was discussed and adopted, the following new work appeared:

§ 4.1. `float` as NTTP

Jorg Brown's [\[P1714\]](#) "NTTP are incomplete without float, double, and long double!" was discussed by EWG ([\[P1714discussion\]](#)), and the reception was favorable (1–14–9–3–3). Proponents want something like

```

template<double Exponent>
double pow(double base);

```

P0732 does not permit this, because `double` does not have strong structural equality. `double` has `partial_ordering`, because of NaN. So Jorg's workaround for C++2a is similar to the `FixedLengthString<6>` hack above: where `FixedLengthString<6>` smuggles a string literal through an array of `char`, Jorg's `AsTemplateArg<double>` smuggles a `double` through an array of `char`.

Two observations:

- It seems that "smuggling contraband through arrays of `char`" is in practice the main use-case for P0732 NTTPs. I have not seen anyone excited about P0732 who isn't planning to use them in this way.
- P0732 is not compatible with EWG's interest in `template<double Exponent>`. `double` will never have strong structural equality, and P0732-based NTTPs will never support types with less-than-strong structural equality.

It seems that EWG is interested in exploring avenues which P0732 cuts off.

In [\[P1714discussion\]](#), one participant is quoted as saying, "My proposal is that we take it [i.e., P0732] out and try again."

§ 4.2. A mangling or serialization operator

Richard Smith writes:

Broadly, I think that attempting to make NTTP identity be the same thing as equality is an evolutionary dead end for C++. They're fundamentally different operations, with different constraints and different goals.

In EWG reflector thread "[isocpp-ext] Can we have float/double as template parameters now?", he informally explored the notion of an overloadable `operator template` which would allow us to pass *any* literal type `T` as an NTTP, as long as it provides a way to "serialize" its value into some serialized form that the compiler knows how to mangle (such as a POD struct), and a way to "deserialize" from that mangled representation back into an object of type `T`.

This approach is premised on the idea that the fundamental building block for NTTPs should not be `==` equality, but rather some sort of *identity* operation. This plays well with the bare fact that `==` equality is already irrelevant to NTTP-identity when the type `T` is an enum type... or a reference type. Consider:


```
constexpr int i = 1;
constexpr int j = 1;

template<const int&>
void foo() {}

static_assert(i == j, "");
static_assert(&foo<i> != &foo<j>, "");
```

Today, in C++17, it is perfectly possible that `static_assert(i == j)` and yet `static_assert(&foo<i> != &foo<j>)`. (Godbolt.)

The importance of this difference between "equality" and "identity" was not widely known during the original discussion of P0732. Had it been known, our approach to NTTPs might have taken a different form.

§ 5. Conclusion

P0732 was premised on an erroneous conflation of "`==` equality" and "NTTP identity." These are similar — but distinguishable — notions. Conflating them causes subtle inconsistencies which will be very hard for any future work in the area to fix.

We should not ship class-typed NTTPs in C++20 without thoroughly exploring the consequences. Once P0732 has appeared in a published standard, it will be too late to fix it.

Incidentally, in hindsight, it was probably a bad idea to allow users to overload `operator==` for enums. It was maybe even a bad idea to allow NTTPs of reference type. But we cannot fix these things (even if we wanted to), because they have already shipped. In this paper, I'm trying to apply foresight (not hindsight) to prevent a feature from shipping *before* we regret it.

The situation with NTTPs in C++17 is subtle and confusing, but at least it's been relatively stable since C++03. P0732 makes some existing issues easier to run into, and causes new issues of its own. Finally, it permanently cuts off potentially fruitful avenues of exploration (such as `float` NTTPs, and user-defined mechanisms for NTTP-identity beyond `==`).

I propose that WG21 remove "class types in non-type template parameters" from C++20, with the expectation that it — or something even better! — may return in C++2b.

Note: This paper (P1837) proposes to remove class-typed NTTPs but leaves `operator<=>` out of scope. Another paper in this mailing, ADAM David Alan Martin's [P1821R0] "Spaceship needs to be grounded," proposes to remove `operator<=>` but leaves class-typed NTTPs out of scope. In Arthur's opinion, it is conceivable to remove both `operator<=>` and class-typed NTTPs (that is, these papers are compatible), or to remove just class-typed NTTPs, but it's unlikely that we could remove just `operator<=>` without also either removing or redesigning class-typed NTTPs.

§ Appendix A: Proposed straw polls

	<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
<i>Revert P0732, with the expectation that it or something better will return in C++2b.</i>	—	—	—	—	—

§ Appendix B: Proposed wording

Note: Arthur will draft wording for the removal of P0732, if called upon to do so. I don't foresee any difficulty with the wording. As far as I know, P0732 is still a "leaf feature" with no library users.

§ References

§ Normative References

[N4700]

Richard Smith. [Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4700.pdf). October 2017. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4700.pdf>

[N4810]

Richard Smith. [Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4810.pdf). March 2019. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4810.pdf>

§ Informative References

[Enums]

Arthur O'Dwyer. [Enums break strong structural equality](https://quuxplusone.github.io/blog/2019/07/04/strong-structural-equality-is-broken/). July 2019. URL: <https://quuxplusone.github.io/blog/2019/07/04/strong-structural-equality-is-broken/>

[N4268]

Richard Smith. [Allow constant evaluation for all non-type template arguments](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4268.html). November 2014. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4268.html>

[P0732]

Jeff Snyder; Louis Dionne. [Class Types in Non-Type Template Parameters](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0732r2.pdf). June 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0732r2.pdf>

[P1714]

Jorg Brown. [NTTP are incomplete without float, double, and long double!](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1714r0.html). June 2019. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1714r0.html>

[P1714discussion]

[Notes from discussion of P1714R0 \(private wiki\)](http://wiki.edg.com/bin/view/Wg21cologne2019/P1714R0-EWG). July 2019. URL: <http://wiki.edg.com/bin/view/Wg21cologne2019/P1714R0-EWG>

[P1821R0]

ADAM David Alan Martin. [The Spaceship Needs to be Grounded: Pull Spaceship from C++20](https://wg21.link/p1821r0). July 2019. URL: <https://wg21.link/p1821r0>